

---

## Отчет по проектному заданию № 1 Моделирование артиллерийской задачи

Выполнил:  
Пахомов Владимир Юрьевич  
Группа: N3150  
Преподаватель:  
Пулькин Н.С

Санкт-Петербург, 2025

1) Описание проектного задания.

В данном проекте будет рассматриваться артиллерийская задача по расчёту траектории полёта снаряда и визуализации траектории движения с помощью языка программирования Python и библиотеки tkinter.

2) Условие:

**3. Моделирование артиллерийской задачи**

На подступах к обороняемому району в пункте расположено орудие, стреляющее в момент времени  $t = 0$  по скоплению противника, разместившегося на расстоянии  $l$  от пункта  $O$ . Скорость снаряда при выходе из канала ствола орудия  $v_0$ . Воздушная среда препятствует движению снаряда с силой, пропорциональной скорости движения. Необходимо воспроизвести траекторию движения снаряда, поражающего цель.

3) Формулы, используемые в расчётах:

$$a_x = \frac{dv_x}{dt} = -k \cdot v_x$$

$$a_y = \frac{dv_y}{dt} = -g - k \cdot v_y$$

$$v_x(t) = v_{0x} \cdot e^{-kt} = v_0 \cdot \cos(\alpha)$$

$$v_y(t) = (v_{0y} + \frac{g}{k}) \cdot e^{-kt} - \frac{g}{k}$$

$$x(t) = \frac{v_{0x}}{k} \cdot (1 - e^{-kt})$$

$$y(t) = \frac{v_{0y} + \frac{g}{k}}{k} \cdot (1 - e^{-kt}) - \frac{g}{k} \cdot t$$

$T$  - время полёта из формулы  $y(T) = 0$

$x(T)$  - дальность полёта из формулы  $x(T)$

4) Краткая теория:

Движение снаряда в воздушной среде описывается баллистической траекторией, на которую влияют:

Начальная скорость  $v_0$  и угол  $\theta$

Сила тяжести  $F_g = m \cdot g$ , направленная вертикально вниз.

Сила сопротивления воздуха  $F_{resist} = -k \cdot v$ , пропорциональна скорости снаряда и направленная противоположно движению.

Уравнения движения выводятся из второго закона Ньютона:

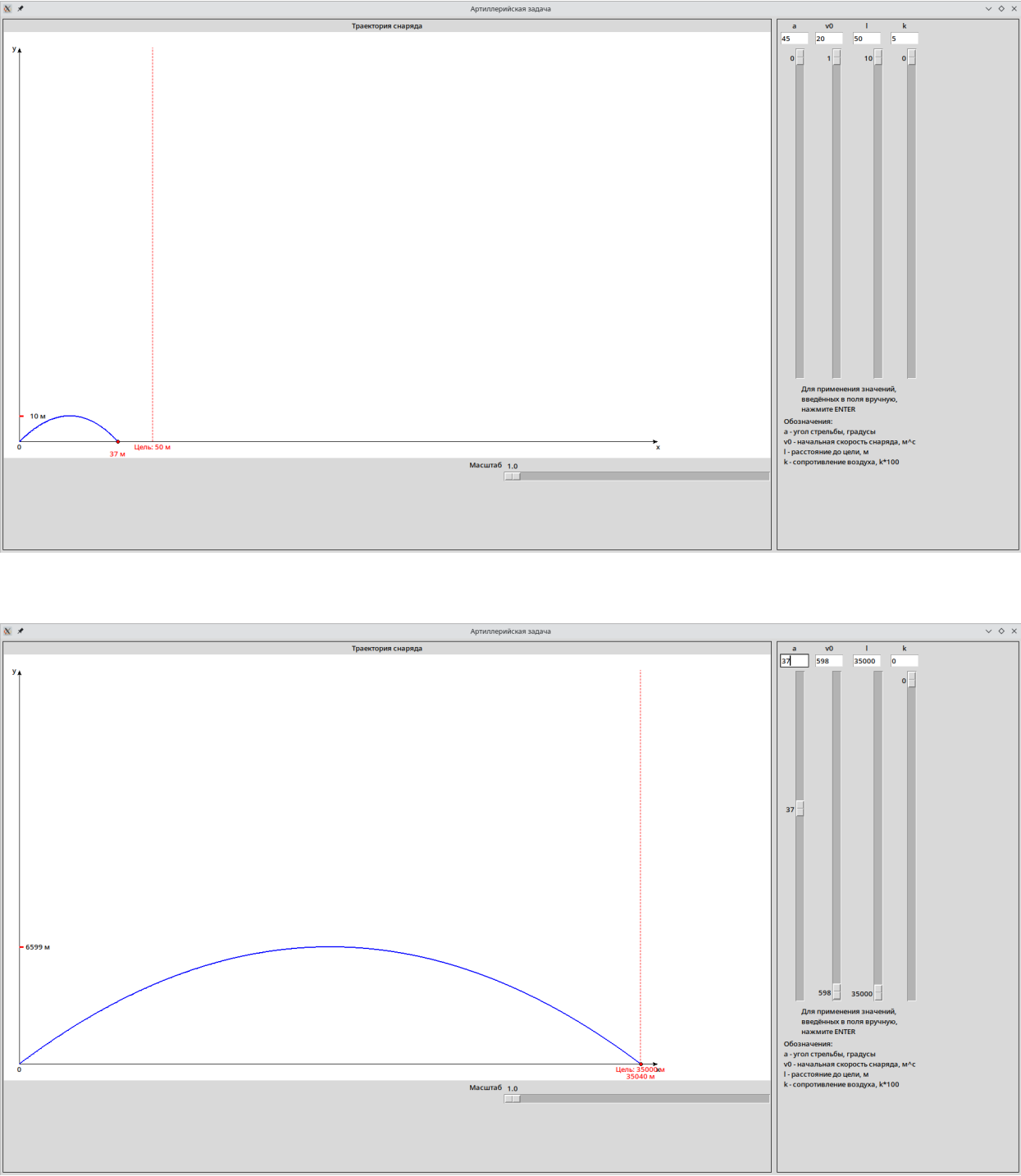
По горизонтали ( $x$ ): сопротивление замедляет снаряд.

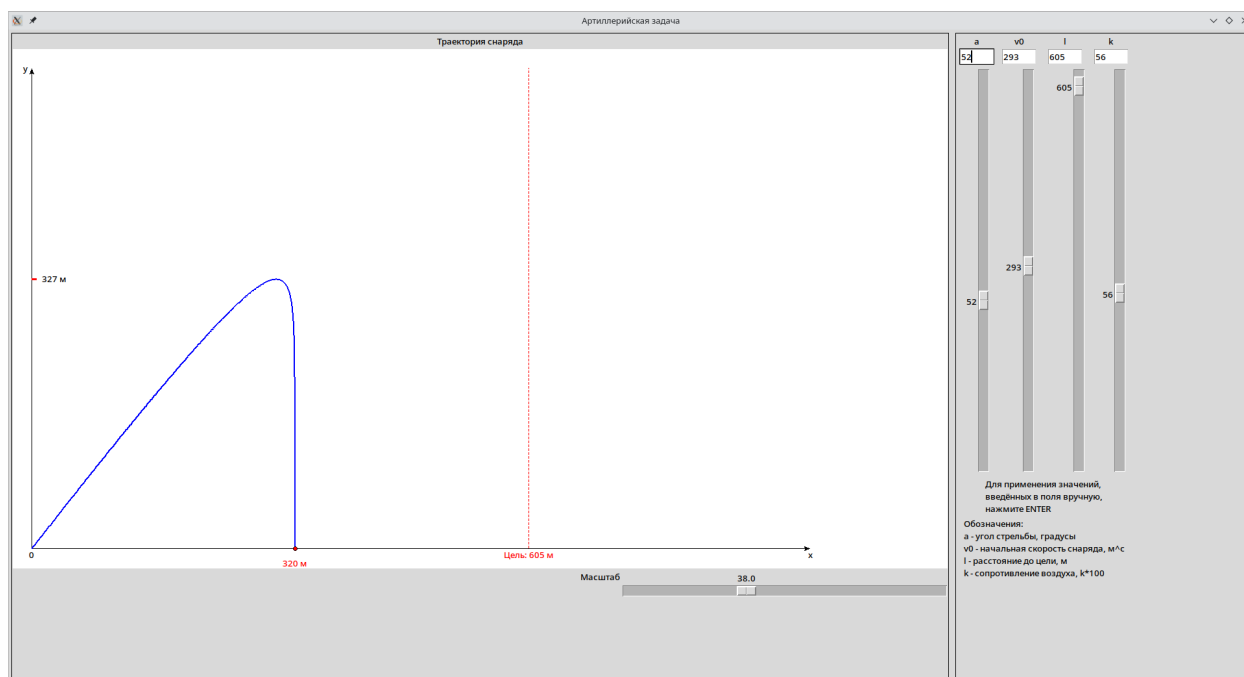
По вертикали ( $y$ ): сопротивление и гравитация совместно уменьшают вертикальную скорость.

5) Описание работы компьютерной модели:

В окне программы слева находится поле, с координатными осями  $x$  и  $y$  и началом отсчета в точке 0. В правой части окна находится секция с четырьмя слайдерами и полями ввода, с помощью которых можно настроить конкретные параметры. При изменении какого-либо параметра в левой части окна обновляется график, соответствующий новым значениям. Также на оси  $y$  показывается максимальная высота подъема (в метрах) снаряда, на оси  $x$  красным кружком показывается расстояние (в метрах), на которое упадёт снаряд, на конце оси  $x$  находится цель с подписью расстояния от начала координат до неё.

6) Примеры работы программы при различных параметрах:





#### 7) Выводы о проделанной работе:

В результате проделанной работы была успешно смоделирована и визуализирована траектория полёта артиллерийского снаряда с учётом сопротивления воздуха. На основе математической модели, основанной на втором законе Ньютона и экспоненциальных решениях дифференциальных уравнений, была реализована компьютерная программа на языке Python с использованием библиотеки tkinter. Интерфейс программы позволяет наглядно изменять параметры модели (угол выстрела, начальная скорость, расстояние до цели и коэффициент сопротивления воздуха) и мгновенно наблюдать, как эти изменения влияют на траекторию полёта.

Полученные результаты подтверждают корректность выбранной физической модели: поведение снаряда при различных параметрах соответствует ожидаемым результатам.

8) Исходный код компьютерной программы:

---

```
python
1  from tkinter import *
2  import numpy as np
3
4
5  def get_angle(value):
6      global alpha_deg
7      alpha_deg = float(value)
8      alpha_deg = min(max(alpha_deg, 1), 90)
9      update_entry_value(entry_angle, round(alpha_deg))
10     draw_trajectory()
11
12 def get_velocity(value):
13     global v0
14     v0 = float(value)
15     v0 = min(max(v0, 1), 600)
16     update_entry_value(entry_velocity, round(v0))
17     draw_trajectory()
18
19 def get_target_distance(value):
20     global l
21     l = float(value)
22     l = min(max(l, 10), 35_000)
23     update_entry_value(entry_target_distance, round(l))
24     draw_trajectory()
25
26 def get_air_resistance(value):
27     global k
28     k = float(value)
29     k = min(max(k, 0), 100)
30     update_entry_value(entry_air_resistance, round(k))
31     k /= 100
32     draw_trajectory()
33
34 def update_entry_value(entry, value):
35     entry.delete(0, END)
36     entry.insert(0, str(value))
37
38 def get_values_from_entries(event):
39     get_angle(entry_angle.get())
40     get_velocity(entry_velocity.get())
41     get_target_distance(entry_target_distance.get())
42     get_air_resistance(entry_air_resistance.get())
43
44 def change_scale(value):
45     global scale
46     scale = float(value) / 30
47     draw_trajectory()
48
49 def calculate_points():
50     alpha = np.radians(alpha_deg)
51     x, y = 0.0, 0.0
52     vx = v0 * np.cos(alpha)
```

```

53     vy = v0 * np.sin(alpha)
54     dt = 0.01
55     pts = [(x, y)]
56     while y >= 0:
57         v = np.hypot(vx, vy)
58         ax = -k * vx
59         ay = -g - k * vy
60         vx += ax * dt
61         vy += ay * dt
62         x += vx * dt
63         y += vy * dt
64         pts.append((x, y))
65     return pts, pts[-1][0]
66
67 def transform_point(x, y):
68     global scale
69     cx = MARGIN + x * scale
70     cy = HEIGHT - (MARGIN + y * scale)
71     return cx, cy
72
73 def draw_trajectory():
74     surface_canvas.delete("all")
75     surface_canvas.create_line(MARGIN, HEIGHT-MARGIN, WIDTH-MARGIN, HEIGHT-MARGIN,
76     ↪ arrow=LAST)
77     surface_canvas.create_line(MARGIN, HEIGHT-MARGIN, MARGIN, MARGIN, arrow=LAST)
78
79     raw, last_x = calculate_points()
80     ys = [y for x, y in raw]
81     max_y_raw = max(ys)
82     max_y_pts = 10**10
83     pts = []
84     for x, y in raw:
85         pts_x, pts_y = transform_point(x, y)
86         pts.append([pts_x, pts_y])
87         if pts_y < max_y_pts:
88             max_y_pts = pts_y
89
90     for (x1, y1), (x2, y2) in zip(pts, pts[1:]):
91         surface_canvas.create_line(x1, y1, x2, y2, fill="blue", width=2)
92
93     tx = MARGIN + 1 * (WIDTH - 2*MARGIN) / 1
94     target_x = transform_point(1, 0)[0]
95     surface_canvas.create_line(target_x, HEIGHT-MARGIN, target_x, MARGIN, fill="red",
96     ↪ dash=(4,2))
97     surface_canvas.create_text(target_x, HEIGHT-MARGIN, text=f"Цель: {round(1)} м",
98     ↪ anchor="n", fill="red")
99     surface_canvas.create_text(pts[-1][0], HEIGHT-MARGIN+13, text=f"{round(last_x)}
100     ↪ м", anchor="n", fill="red")
101     surface_canvas.create_oval(pts[-1][0]-3, HEIGHT-MARGIN-3, pts[-1][0]+3,
102     ↪ HEIGHT-MARGIN+3, fill="red")
103     surface_canvas.create_line(MARGIN, max_y_pts, MARGIN+8, max_y_pts, width=3,
104     ↪ fill="red")
105     surface_canvas.create_text(MARGIN+35, max_y_pts-10, text=f"{round(max_y_raw)} м",
106     ↪ anchor="n")

```

```

100     surface_canvas.create_text(MARGIN-10, MARGIN, text="y")
101     surface_canvas.create_text(WIDTH-MARGIN, HEIGHT-MARGIN, text="x", anchor="n")
102     surface_canvas.create_text(MARGIN, HEIGHT-MARGIN, text="0", anchor="n")
103
104
105     l = 50           # расстояние до цели (м)
106     v0 = 20          # начальная скорость снаряда (м/с)
107     g = 9.81         # ускорение свободного падения (м/с^2)
108     k = 0.05         # коэффициент сопротивления воздуха
109     alpha_deg = 45   # угол в градусах
110     scale = 5        # масштаб
111
112     root = Tk()
113     root.geometry("1600x900")
114     root.title("Артиллерийская задача")
115     root.columnconfigure(index=0, weight=2)
116     root.columnconfigure(index=1, weight=2)
117     root.rowconfigure(index=0, weight=1)
118
119     surface = Frame(borderwidth=1, relief=SOLID)
120     surface_hint_label = Label(surface, text="Траектория снаряда")
121     surface_scale_label = Label(surface, text="Масштаб")
122     surface_scale_slider = Scale(surface, command=change_scale, from_=1, to=100,
123     ↪ resolution=0.1, orient='horizontal', length=500)
124
125     WIDTH = 1260
126     HEIGHT = 800
127     MARGIN = 30
128     surface_canvas = Canvas(surface, bg="white", width=WIDTH, height=HEIGHT)
129     surface_canvas.create_line(MARGIN, HEIGHT - MARGIN, WIDTH - MARGIN, HEIGHT - MARGIN,
130     ↪ arrow=LAST)
131     surface_canvas.create_line(MARGIN, HEIGHT - MARGIN, MARGIN, MARGIN, arrow=LAST)
132
133     surface.grid(row=0, column=0, sticky=NSEW, padx=5, pady=5)
134     surface_hint_label.pack()
135     surface_canvas.pack(fill=BOTH)
136     surface_scale_slider.pack(side=RIGHT, anchor=N)
137     surface_scale_label.pack(side=RIGHT, anchor=N)
138
139     sliders = Frame(borderwidth=1, relief=SOLID)
140     slider_angle = Scale(sliders, command=get_angle, from_=0, to=90, length=620)
141     slider_velocity = Scale(sliders, command=get_velocity, from_=1, to=600, length=620)
142     slider_target_distance = Scale(sliders, command=get_target_distance, from_=10,
143     ↪ to=35_000, length=620)
144     slider_air_resistance = Scale(sliders, command=get_air_resistance, from_=0, to=100,
145     ↪ length=620)
146
147     entry_angle = Entry(sliders, width=6)
148     entry_velocity = Entry(sliders, width=6)
149     entry_target_distance = Entry(sliders, width=6)
150     entry_air_resistance = Entry(sliders, width=6)
151     entry_angle.insert(0, str(alpha_deg))
152     entry_velocity.insert(0, str(v0))

```

```

150 entry_target_distance.insert(0, str(1))
151 entry_air_resistance.insert(0, str(round(k*100)))
152
153 label_angle = Label(sliders, text="a")
154 label_velocity = Label(sliders, text="v0")
155 label_target_distance = Label(sliders, text="l")
156 label_air_resistance = Label(sliders, text="k")
157
158 sliders.grid(row=0, column=1, sticky=NSEW, padx=5, pady=5)
159 slider_angle.grid(row=2, column=0, pady=5)
160 slider_velocity.grid(row=2, column=1, pady=5)
161 slider_target_distance.grid(row=2, column=2, pady=5)
162 slider_air_resistance.grid(row=2, column=3, pady=5)
163
164 entry_angle.grid(row=1, column=0)
165 entry_velocity.grid(row=1, column=1)
166 entry_target_distance.grid(row=1, column=2)
167 entry_air_resistance.grid(row=1, column=3)
168
169 label_angle.grid(row=0, column=0)
170 label_velocity.grid(row=0, column=1)
171 label_target_distance.grid(row=0, column=2)
172 label_air_resistance.grid(row=0, column=3)
173
174 label_tip = Label(sliders, text="Для применения значений,\nвведённых в поля
↪ вручную,\nнажмите ENTER", justify="left")
175 legend_text = """Обозначения:
176 a - угол стрельбы, градусы
177 v0 - начальная скорость снаряда, м^с
178 l - расстояние до цели, м
179 k - сопротивление воздуха, k*100
180 """
181 label_legend = Label(sliders, text=legend_text, justify="left")
182 label_tip.grid(row=3, column=0, columnspan=4)
183 label_legend.grid(row=4, column=0, columnspan=4, padx=10)
184
185 root.bind("<Return>", get_values_from_entries)
186
187 draw_trajectory()
188
189 root.mainloop()

```

---